

Final Project

Danny Halovanic

2023-05-11

Contents

Introduction	2
Description of Model	2
Model Components	2
Numerical Experiment	3
Results	4
Appendix	4

Introduction

In this project, we will predict the occurrence of Type II Diabetes. Specifically, we will use the data to predict whether a patient has diabetes. The data for our project is from the National Institute of Diabetes and Digestive and Kidney Diseases. All of the patients are Pima Indian females of 21 years and older. The raw data can be obtained from <https://raw.githubusercontent.com/pengdsci/MAT325/main/w12/diabetes.csv>

Description of Model

In order to determine if a certain patient has diabetes, we must use a model. The probability model we will be using is

$$P[d = 1] = \frac{e^{\beta_0 + \beta_1 x + \beta_2 y + \beta_3 z + \dots}}{1 + e^{\beta_0 + \beta_1 x + \beta_2 y + \beta_3 z + \dots}}$$

where d is diabetes status (1=diabetes and 0=no diabetes). $x, y, z, \dots$ are the risk factors of diabetes.

Additionally, the objective function that needs to be maximized is

$$\mathbf{LL}(\beta_0, \beta_1, \beta_2, \dots) = - \sum_{i=1}^n \ln[1 + e^{\beta_0 + \beta_1 x_i + \beta_2 y_i + \dots}] + \sum_{i=1}^n d_i(\beta_0 + \beta_1 x_i + \beta_2 y_i + \dots)$$

Model Components

In this section, we will be deriving the gradient vector and the Hessian Matrix. Since we have 2 independent variables in this model, we will have 3 components to our gradient vector.

$$\begin{aligned}\frac{\partial LL}{\partial \beta_0} &= - \sum_{i=1}^n \frac{e^{\beta_0 + \beta_1 x_i + \beta_2 y_i}}{1 + e^{\beta_0 + \beta_1 x_i + \beta_2 y_i}} + \sum_{i=1}^n d_i = 0 \\ \frac{\partial LL}{\partial \beta_1} &= - \sum_{i=1}^n \frac{e^{\beta_0 + \beta_1 x_i + \beta_2 y_i}}{1 + e^{\beta_0 + \beta_1 x_i + \beta_2 y_i}} x_i + \sum_{i=1}^n x_i d_i = 0 \\ \frac{\partial LL}{\partial \beta_2} &= - \sum_{i=1}^n \frac{e^{\beta_0 + \beta_1 x_i + \beta_2 y_i}}{1 + e^{\beta_0 + \beta_1 x_i + \beta_2 y_i}} y_i + \sum_{i=1}^n y_i d_i = 0\end{aligned}$$

Since we are using 2 independent variables in this model, the Hessian Matrix must be a 3x3 matrix. The Hessian Matrix is defined as

$$H(\beta_0, \beta_1, \beta_2) = \begin{bmatrix} \mathbf{LL}_{\beta_0\beta_0} & \mathbf{LL}_{\beta_0\beta_1} & \mathbf{LL}_{\beta_0\beta_2} \\ \mathbf{LL}_{\beta_1\beta_0} & \mathbf{LL}_{\beta_1\beta_1} & \mathbf{LL}_{\beta_1\beta_2} \\ \mathbf{LL}_{\beta_2\beta_1} & \mathbf{LL}_{\beta_2\beta_1} & \mathbf{LL}_{\beta_2\beta_2} \end{bmatrix}$$

We can derive the second derivatives. Since the second sum is a constant value, that derivative is equal to 0 so we only need to focus on the first sum. Additionally, We can say that $\mathbf{LL}_{ab} = \mathbf{LL}_{ba}$ since Clairaut's Theorem tells us the mixed partial derivatives are equal

$$\begin{aligned} \mathbf{LL}_{\beta_0\beta_0} &= - \sum_{i=1}^n \frac{e^{\beta_0+\beta_1x_i+\beta_2y_i}}{(1 + e^{\beta_0+\beta_1x_i+\beta_2y_i})^2} \\ \mathbf{LL}_{\beta_0\beta_1} &= - \sum_{i=1}^n \frac{e^{\beta_0+\beta_1x_i+\beta_2y_i}}{(1 + e^{\beta_0+\beta_1x_i+\beta_2y_i})^2} x_i = \mathbf{LL}_{\beta_1\beta_0} \\ \mathbf{LL}_{\beta_0\beta_2} &= - \sum_{i=1}^n \frac{e^{\beta_0+\beta_1x_i+\beta_2y_i}}{(1 + e^{\beta_0+\beta_1x_i+\beta_2y_i})^2} y_i = \mathbf{LL}_{\beta_2\beta_0} \\ \mathbf{LL}_{\beta_1\beta_1} &= - \sum_{i=1}^n \frac{e^{\beta_0+\beta_1x_i+\beta_2y_i}}{(1 + e^{\beta_0+\beta_1x_i+\beta_2y_i})^2} x_i^2 \\ \mathbf{LL}_{\beta_1\beta_2} &= - \sum_{i=1}^n \frac{e^{\beta_0+\beta_1x_i+\beta_2y_i}}{(1 + e^{\beta_0+\beta_1x_i+\beta_2y_i})^2} x_i y_i = \mathbf{LL}_{\beta_2\beta_1} \\ \mathbf{LL}_{\beta_2\beta_2} &= - \sum_{i=1}^n \frac{e^{\beta_0+\beta_1x_i+\beta_2y_i}}{(1 + e^{\beta_0+\beta_1x_i+\beta_2y_i})^2} y_i^2 \end{aligned}$$

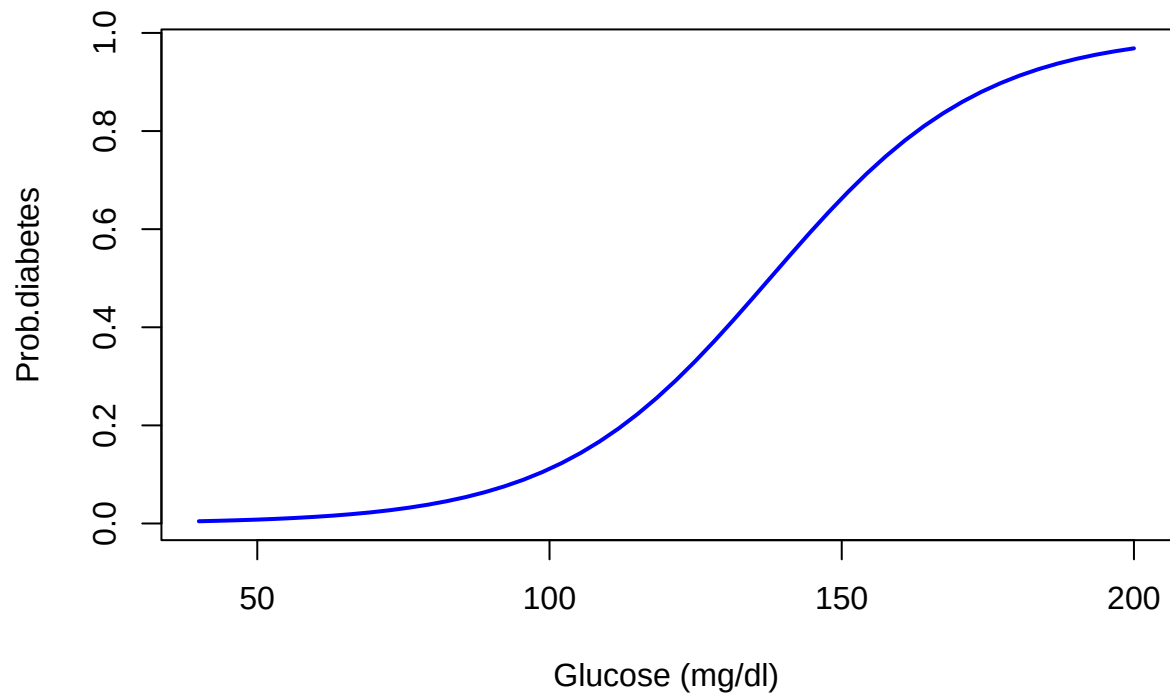
Numerical Experiment

We can plot BMI and Glucose with probability of Diabetes

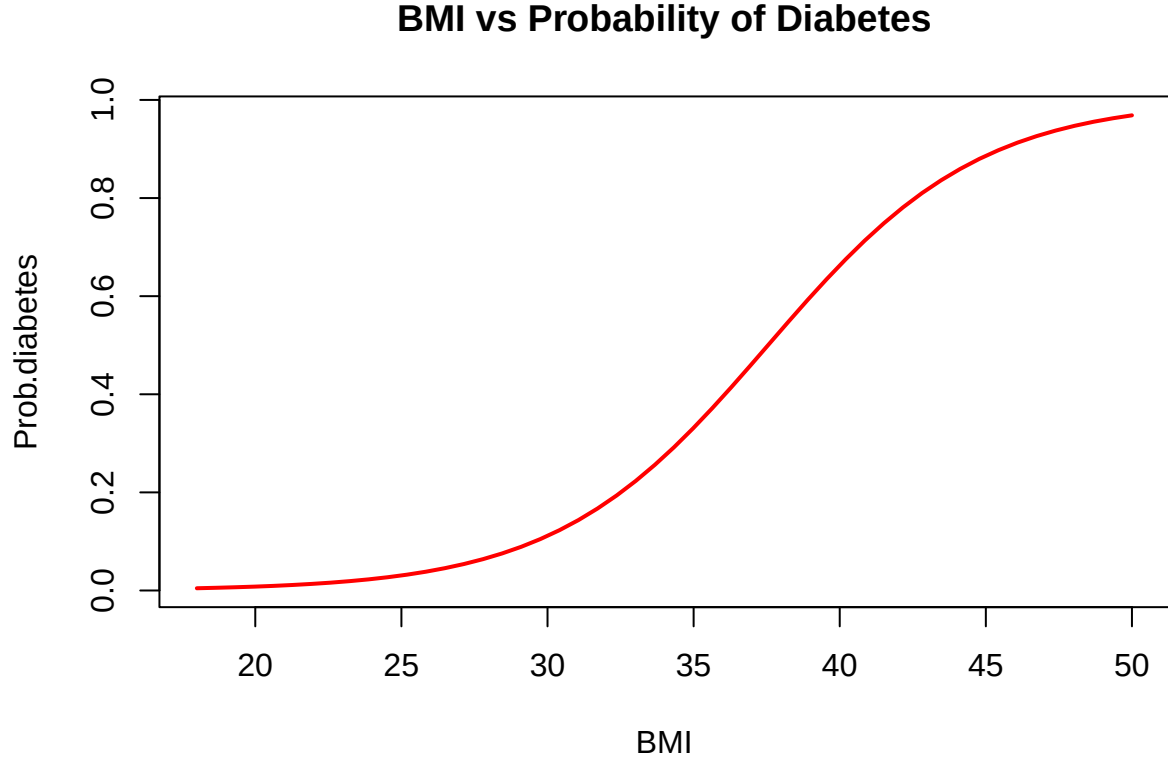
```
Glucose=seq(40,200,length.out=50)
bmi=seq(18,50,length.out=50)
E =exp(-8.3015+0.0718*(bmi)+0.0407*Glucose) # values derived from the R code in appendix
Prob.diabetes = E/(1 + E) # the probabily equation from the model

plot(Glucose, Prob.diabetes, type="l", lwd=2, col="blue", xlab="Glucose (mg/dl)",main="Glucose vs Probab
```

Glucose vs Probability of Diabetes



```
plot(bmi, Prob.diabetes, type="l", lwd=2, col="red", xlab="BMI", main="BMI vs Probability of Diabetes")
```



Results

From the R Code in the appendix, we get $\beta_0 = -8.3015$, $\beta_1 = 0.0718$, and $\beta_2 = 0.0407$. Therefore, our model, with BMI and Glucose Levels being x and y respectively can be represented by the following:

$$P[Y = Diabetes] = \frac{e^{-8.3015+0.0718x+0.0407y}}{1 + e^{-8.3015+0.0718x+0.0407y}}$$

For example, if we have a patient whose BMI is 30 and whose blood glucose level is 200mg/dl, their probability of having type 2 Diabetes can be predicted.

$$P[Y = Diabetes] = \frac{e^{-8.3015+0.0718(30)+0.0407(200)}}{1 + e^{-8.3015+0.0718(30)+0.0407(200)}} \approx 0.8800 \approx 88\%$$

The mathematical model demonstrates that the higher the BMI for a patient, the odds of having type II Diabetes increases and the higher the blood glucose level, the odds of having type II Diabetes increases. However, this mathematical model only uses those variables and does not include other lifestyle factors or genetic factors. Additionally, the model is limited since the data does not include men or any other groups.

Appendix

Here is the R code for calculating the Hessian Matrix:

```

JH = function(x,x2, y, a0, a1, a2){
  # x = x-coordinates
  # x2
  # y = y-coordinates
  # a0 = alpha_0, a1 = alpha_1 a2=alpha_2
  E = exp(a0+a1*x+a2*x2)
  L0 = -sum(E/(1+E)) + sum(y)
  L1 = -sum(x*E/(1+E)) + sum(x*y)
  L2 = -sum(x2*E/(1+E))+ sum(x2*y)
  LL00 = -sum(E/(1+E)^2)
  LL01 = -sum(x*E/(1+E)^2)
  LL02= -sum(x2*E/(1+E)^2)
  LL10 = LL01
  LL11 = -sum(x^2*E/(1+E)^2)
  LL12 = -sum(x2*x*E/(1+E)^2)
  LL20=LL02
  LL21=LL12
  LL22= -sum(x2^2/(1+E)^2)
  LV = c(L0, L1,L2)
  J = matrix(c(LL00, LL01,LL02, LL10, LL11,LL12,LL20,LL21,LL22), ncol =3, byrow = TRUE)
  list(LV = LV, Jacob = J)
}

```

```

Newton.Alg=function(x,x2, y, ini.val, tol, maxit=100, trace = TRUE){
  ### initialization
  err = 1
  i = 1
  sol.mtx = matrix(0, nrow=maxit, ncol=length(ini.val))
  err.vec = rep(0, maxit)
  fn.mtx = matrix(0, nrow=maxit, ncol=length(ini.val))
  while(err > tol && i < maxit){
    ### initialization
    JnFun = JH(x,x2,y, a0 = ini.val[1],a1 = ini.val[2], a2=ini.val[3])
    JMatrix = JnFun$Jacob
    fn.vec = JnFun$LV
    if(det(JMatrix) == 0){
      cat("\n\n The Hessian matrix is singular!")
      break
    }
    h = - solve(JMatrix)%*%fn.vec
    new.val = ini.val + h
    err=max(abs(h))
    ## store intermediate outputs
    err.vec[i] = err
    sol.mtx[i,] = as.vector(new.val)
    fn.mtx[i,] = JH(x,x2,y, a0 = ini.val[1],a1 = ini.val[2], a2=ini.val[3])$LV
    ## updating the root and the iteration ID
    ini.val=new.val
    i = i + 1
  }
  id = which(err.vec==0)[1]-1 # locate the starting rows with all zero cells
}

```

```

## Determinant of Hessian
SOL = sol.mtx[id,]
D = det(JH(x, x2,y, a0 = SOL[1], a1 = SOL[2], a2=SOL[3])$Jacob)
#D = det(Jacobian(sol.mtx[id,]))
fxx = (JH(x, x2,y, a0 = SOL[1], a1 = SOL[2], a2=SOL[3])$Jacob)[1,1]
##
if(trace ==TRUE){
  list(solution = sol.mtx[1:id,], error = err.vec[1:id], fn.values = fn.mtx[1:id,])
} else {
  if(D > 0 & fxx > 0) extreme = "local minimum"
  if(D > 0 & fxx < 0) extreme = "local maximum"
  if(D < 0 ) extreme = "saddle point"
  list(iterations = id, solution = sol.mtx[id,],
       D = D,
       fxx = fxx,
       extreme = extreme,
       error = err.vec[id],
       fn.values = fn.mtx[id,])
}
}

```

The R code below runs the Newton's Method from above to generate initial values:

```

diabetes = read.csv("https://raw.githubusercontent.com/pengdsci/MAT325/main/w12/diabetes.csv")
diabeticStatus = diabetes$Outcome      # y-variable: 1 = diabetes, 0 = no-diabetes
BMI = diabetes$BMI # x-variable (risk factor)
Glucose=diabetes$Glucose # the other x-variable (risk factor)
#glm(diabeticStatus ~ BMI, family = binomial(link = logit))
Newton.Alg(x=BMI, x2=Glucose, y=diabeticStatus, ini.val=c(0,0,0), tol = 10-8, maxit=500, trace = FALSE)

```

```

## $iterations
## [1] 467
##
## $solution
## [1] -8.30145952  0.07179355  0.04071319
##
## $D
## [1] -210518546233
##
## $fxx
## [1] -60.45291
##
## $extreme
## [1] "saddle point"
##
## $error
## [1] 9.744794e-09
##
## $fn.values
## [1] -5.684342e-14 -2.728484e-12  9.999227e-05

```